1 Basic Definitions

Learning: Find a model f that maps from inputs to outputs given a labeled training dataset.
Inference: Use a model f to predict output y given new example $\vec{x}$
Model: Function f mapping input $\vec{x}$ to output prediction y.
Model family: Set of models that share the same structure but differ in parameters (e.g. layers in a NN)
Accuracy: $\frac{\sum_{i=1}^{N} \mathbb{I}\left[t^{(i)}=y^{(i)}\right]}{N}$
Parameter: Internal variable of model learned from data whilst training
Hyperparameter: Settings chosen before training that controls how the learning process works.
Parametric model: Any model with the parameter counts NEVER being $\Omega(N)$ (invariant to the data count)
Precision: TPR= $\frac{TP}{TP+FN} = p$ (actually +|pred+)
Recall: $\frac{TP}{TP+FN} = p$ (pred +|actually +)

1.1 KNN

Find $t^{(c)}$ corresponding to
$\vec{x}^{(c)} = \arg\min_{\vec{x}^{(i)} \in \text{train set}} \text{distance}\left(\vec{x}^{(i)}, \vec{x}\right)$
Adv: Simple, no training, works for reg. and class, adapts easily to new data.
Disadvantage: Bad w/ high dims, sensitive to scaling, slow for large datasets, must store logs proportional to data. Make k odd or chose a tiebreaker.

2 Decision Trees

Large trees overfit, small trees underfit, trace the tree on inference. $H(X) = -\sum_{x \in X} p(x) \log_2 p(x)$,
$H(X|Y=y) = -\sum_{x \in X} p(x|Y=y) \log_2 p(x|Y=y)$.
Expected conditional entropy:
$H(X|Y) = -\sum_{y \in Y} p(y) \sum_{x \in X} p(x|y) \log_2 p(x|y)$,
$IG(X|Y) = H(X) - H(X|Y)$
Properties: Entropy nonnegative, $H(Y|X) \leq H(Y)$, Y, X independent $\Rightarrow IG(Y|X) = 0$, fully determinant rids entropy.
Splitting: Keep splitting until all examples have same label, when no features left, when depth limit is reached (hyperparam), when loss stops decreasing enough (hyperparam)

3 Linear Regression

Loss measures how off **one feature** is, cost measures **all**. $\mathcal{L}(y,t) = \frac{1}{2}(y-t)^2$, $\mathcal{E}(\vec{w}) = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}\left(y^{(i)}, t^{(i)}\right)$.
Hence, for linear model $\vec{y} = W\vec{x}$.

- $\mathcal{E}(\vec{w}) = \frac{1}{2N}\sum_{i=1}^{N}\left(\vec{w}^\top \vec{x} - t^{(i)}\right)^2$
- $\mathcal{E}(\vec{w}) = \frac{1}{2N}\left(\mathbf{X}\vec{w} - \vec{t}\right)^\top \left(\mathbf{X}\vec{w} - \vec{t}\right)$

Gradients:

- $\frac{\partial \mathcal{E}}{\partial w_j} = \frac{1}{N}\sum_{i=1}^{N}\left(\vec{w}^\top \vec{x}^{(i)} - t^{(i)}\right) x_j^{(i)}$
- $\nabla_{\vec{w}} \mathcal{E}(\vec{w}) = \frac{1}{N}\sum_{i=1}^{N}\left(\vec{w}^\top \vec{x}^{(i)} - t^{(i)}\right) \vec{x}^{(i)}$
- $\nabla_{\vec{w}} \mathcal{E}(\vec{w}) = \frac{1}{N}\mathbf{X}^\top\left(\mathbf{X}\vec{w} - \vec{t}\right)$

W/ bias: $\frac{\partial \mathcal{E}(\vec{w})}{\partial w_j} = \frac{1}{N}\sum_{i=1}^{N}\left(\vec{w}^\top \vec{x}^{(i)} + b - t^{(i)}\right) x_j^{(i)}$,
$\frac{\partial \mathcal{E}(\vec{w})}{\partial b} = \frac{1}{N}\left(\mathbf{X}\vec{w} + b - \vec{t}\right)$
Regularizer: $\frac{\lambda}{2}\sum_{j=0}^{D} w_j^2$, $\nabla_{\vec{w}}\left(\frac{\lambda}{2}\vec{w}^\top \vec{w}\right) = \lambda \vec{w}$
Stochastic GD: Randomly pick a training example, very noisy, $\vec{w} \leftarrow \vec{w} - \alpha \frac{\partial \mathcal{L}}{\partial \vec{w}}$. Mini-batch GD falls in-between. Batch size is a hyperparameter

4 Logistic Regression

- $z = \mathbf{w}^\top \mathbf{x}$, $y = \mathbb{I}[z \geq 0]$

Loss functions:

- 0-1 $\mathcal{L}_{0-1}(y,t) = \mathbb{I}[y \neq t]$. Hard to optimize
- $\mathcal{L}_{SE}(z,t) = \frac{1}{2}(z-t)^2$. Large loss for v. correct
- $y = \sigma(z) = \frac{1}{1+e^{-z}}$, $\mathcal{L}_{SE}(y,t) = \frac{1}{2}(y-t)^2$, prediction always [0,1], ∇ small at v. large magnitudes
- Cross entropy:

$$\mathcal{L}_{CE}(y,t) = \begin{cases} -t\log(y) & t=1 \\ -(1-t)\log(1-y) & t=0 \end{cases}$$

CE loss derivation, $z = \vec{w}^T \vec{x}$. $y = \sigma(z) = \frac{1}{1+e^{-z}}$.
$\sigma'(z) = \sigma(z)(1-\sigma(z))$. $\frac{\partial y}{\partial z} = y(1-y)$.
$\mathcal{L}_{CE}(y,t) = -t\log(y) = -(1-t)\log(1-y)$.
GD update: $\mathcal{E}(\vec{w}) = \frac{1}{N}\sum_{i=1}^{N}\mathcal{L}_{CE}\left(y^{(i)}, t^{(i)}\right)$.

- $\frac{\partial \mathcal{L}_{CE}}{\partial w_j} = \frac{\partial \mathcal{L}_{CE}}{\partial y}\frac{\partial y}{\partial z}\frac{\partial z}{\partial w_j}$
- $= \left(-\frac{t}{y} + \frac{1-t}{1-y}\right) y(1-y) x_j = (y-t)x_j$
- Vectorized: $\vec{w} \leftarrow \vec{w} - \frac{\alpha}{N}\mathbf{X}^\top\left(\mathbf{X}\vec{w} - \vec{t}\right)$

4.1 Softmax Gradient

The softmax model is

- $\mathbf{z} = \mathbf{W}\mathbf{x}$ $\quad \mathbf{y} = \text{softmax}(\mathbf{z})$
- $y_k = \frac{\exp(z_k)}{\sum_{m=1}^{K}\exp(z_m)}$
- $\mathcal{L}(\mathbf{y}, \mathbf{t}) = -\mathbf{t}^\top \log(\mathbf{y})$

Derive: $\frac{\partial L}{\partial \mathbf{w}_m}$. Derivation:

- $L = -\sum_{k=1}^{K} t_k \log(y_k)$
- $\frac{\partial L}{\partial y_k} = -\frac{t_k}{y_k}$
- $k \neq m \Rightarrow \frac{\partial y_k}{\partial z_m} = \frac{\partial}{\partial z_m}\left(\frac{\exp(z_k)}{\sum_{n=1}^{K} z_n}\right)$
- $= \frac{-\exp(z_k)\exp(z_m)}{\left(\sum_{n=1}^{K}\exp(z_n)\right)^2} = -y_k y_m$
- $k = m \Rightarrow \frac{\partial y_k}{\partial z_k} = \frac{\partial}{\partial z_k}\left(\frac{\exp(z_k)}{\sum_{n=1}^{K}\exp(z_n)}\right)$
- $= \frac{\exp(z_k)\left(\left(\sum_{n=1}^{K}\exp(z_n)\right) - \exp(z_k)\right)}{\left(\sum_{n=1}^{K}\exp(z_n)\right)^2}$
- $= y_k(1-y_k)$
- $\frac{\partial y_k}{\partial z_m} = \begin{cases} -y_k y_m & \text{if } k \neq m \\ y_k(1-y_k) & \text{if } k = m \end{cases}$
- $\frac{\partial L}{\partial z_m} = \sum_{k=1}^{K}\frac{\partial L}{\partial y_k}\frac{\partial y_k}{\partial z_m}$
- $= \sum_{k=1}^{K}\begin{cases} t_k y_m & \text{if } k \neq m \\ t_k(y_k - 1) & \text{if } k = m \end{cases}$
- $= t_1 y_m + \ldots + t_m(y_m - 1) + \ldots + t_K y_m$

Since $\mathbf{t}$ is one-hot: if $t_m = 1$, term left in sum is $t_m(y_m - 1) = y_m - 1 = y_m - t_m$. If $t_k = 1$ where $k \neq m$, then $t_m = 0 \Rightarrow t_k y_m = y_m = y_m - t_m$. Either way, we have that $\frac{\partial L}{\partial z_m} = y_m - t_m$. Hence:
$\frac{\partial L}{\partial \mathbf{w}_m} = \frac{\partial L}{\partial z_m}\frac{\partial z_m}{\partial \mathbf{w}_m} = (y_m - t_m)\,\mathbf{x}$

4.2 Softmax K=2 Proof

$K = 2$, $y_k = \frac{e^{z_k}}{\sum_{m=1}^{K} e^{z_m}}$ correspond to $y = \frac{1}{1+e^{-z}}$.

- $y_1 = \frac{e^{z_1}}{e^{z_1}+e^{z_2}} = \frac{e^{z_1}}{e^{z_1}(1+e^{z_2-z_1})} = \frac{1}{1+e^{-(z_1-z_2)}}$

5 NNs

An MLP is a feed-forward network composed of an input layer, one or more hidden layers, and output layers.
AND: $x_1 + x_2 - 1.5$, OR: $x_1 + x_2 - 0.5$
XOR: $(x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$, alt:
$(x_1 \vee x_2) \wedge (\neg(x_1 \wedge x_2))$
Brute forcing:
$\left[-N1_{\text{TRUE}} + 0.5, -999, -999, 1, 1, -999\right]$

6 Backprop

$$\begin{tikzcd}
\mathbf{a}^0 \arrow[r] & \mathbf{z}^1 \arrow[r] & \mathbf{a}^1 \arrow[r] & \mathbf{z}^2 \arrow[r] & \mathbf{a}^2 \arrow[r] & z^3 \arrow[r] & a^3 \arrow[r] & \mathcal{L} \\
\mathbf{W}^1 \arrow[ur] & \mathbf{b}^1 \arrow[u] & \mathbf{W}^2 \arrow[ur] & \mathbf{b}^2 \arrow[u] & \mathbf{W}^3 \arrow[ur] & b^3 \arrow[u] & &
\end{tikzcd}$$

Backpropagation efficiently computes all the derivatives in a single pass by reusing intermediate results.
Forward pass. System computes and stores pre-activations z and activations a for every layer, proceeding from the input to the output. Needed b/c these values must be used later
Backward pass. Algorithm starts by computing the gradient of the loss function with respect to the output layer's pre-activations. It recursively computes gradients for hidden layers, moving backwards from the output to the input. It uses the multivariate chain rule to propagate error terms through the network. $\nabla_{\mathbf{x}}(\mathbf{A}\mathbf{x}) = \mathbf{A}$,
$\nabla_{\mathbf{x}}(\mathbf{x} \odot \mathbf{a}) = \text{diag}(\mathbf{a})$, $\nabla_{\mathbf{x}}\mathbf{f}(\mathbf{x}) = \text{diag}\left(\mathbf{f}'(\mathbf{x})\right)$
elementwise, $\nabla_{\mathbf{x}}(\mathbf{f}(\mathbf{x}))^\top A(\mathbf{f}(\mathbf{x})) = 2\mathbf{f}'(\mathbf{x})^\top A\mathbf{f}(\mathbf{x})$ and $\nabla_{\mathbf{x}} A\mathbf{f}(\mathbf{x}) = A\mathbf{f}'(\mathbf{x})$.

6.1 Backprop Vectorizer

- **PRE-ACT:** $z_j^{(1)} = \sum_{i=1}^{D^{(0)}} \mathbf{W}_{ji}^{(1)} a_i^{(0)} + b_j^{(1)}$
  - $\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{a}^{(0)} + \mathbf{b}^{(1)}$
  - $\mathbf{Z}^{(1)} = \mathbf{A}^{(0)}\left(\mathbf{W}^{(1)}\right)^\top + \mathbf{1}_N\left(\mathbf{b}^{(1)}\right)^\top$
- **POST-ACT:** $a_j^{(1)} = \text{ReLU}\left(z_j^{(1)}\right)$
  - $\mathbf{a}^{(1)} = \text{ReLU}\left(\mathbf{z}^{(1)}\right)$, $\mathbf{A}^{(1)} = \text{ReLU}\left(\mathbf{Z}^{(1)}\right)$
- **SOFTMAX:** $a_k^{(2)} = \frac{e^{z_k^{(2)}}}{\sum_{k'=1}^{D^{(2)}} e^{z_{k'}^{(2)}}}$
  - $\mathbf{a}^{(2)} = \frac{\exp\left(\mathbf{z}^{(2)}\right)}{\mathbf{1}_{D^{(2)}}^\top \exp\left(\mathbf{z}^{(2)}\right)}$
  - $\mathbf{A}^{(2)} = \frac{\exp\left(\mathbf{Z}^{(2)}\right)}{\exp\left(\mathbf{Z}^{(2)}\right)\mathbf{1}_{D^{(2)}}}$
- **CE:** $\mathcal{L}\left(\mathbf{a}^{(2)}, \mathbf{t}\right) = -\sum_{k=1}^{D^{(2)}} t_k \log\left(a_k^{(2)}\right)$
  - $\mathcal{L}\left(\mathbf{a}^{(2)}, \mathbf{t}\right) = -\mathbf{t}^\top \log\left(\mathbf{a}^{(2)}\right)$
  - $\mathcal{L} = -\frac{1}{N}\mathbf{1}_N^\top\left(T \odot \log \mathbf{A}^{(2)}\right)\mathbf{1}_{D^{(2)}}$
- ∇ **CE:** $\frac{\partial \mathcal{L}}{\partial a_k^{(2)}} = -\frac{t_k}{a_k^{(2)}}$ *(look at the softmax grad)*
  - $\nabla_{\mathbf{a}^{(2)}}\mathcal{L} = -\frac{\mathbf{t}}{\mathbf{a}^{(2)}}$, $\nabla_{\mathbf{A}^{(2)}}\mathcal{L} = -\frac{\mathbf{T}}{\mathbf{A}^{(2)}}$
- ∇ **SOFTMAX:** $\frac{\partial \mathcal{L}}{\partial z_k^{(2)}} = a_k^{(2)} - t_k$
  - $\nabla_{\mathbf{z}^{(2)}}\mathcal{L} = \mathbf{a}^{(2)} - \mathbf{t}$
  - $\nabla_{\mathbf{Z}^{(2)}}\mathcal{L} = \frac{1}{N}\left(\mathbf{A}^{(2)} - \mathbf{T}\right)$
- ∇ **WEIGHTS:** $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{kj}^{(2)}} = \frac{\partial \mathcal{L}}{\partial z_k^{(2)}} a_j^{(1)}$
  - $\nabla_{\mathbf{W}^{(2)}}\mathcal{L} = \left(\nabla_{\mathbf{z}^{(2)}}\mathcal{L}\right)\left(\mathbf{a}^{(1)}\right)^\top$
  - $\nabla_{\mathbf{W}^{(2)}}\mathcal{L} = \left(\nabla_{\mathbf{Z}^{(2)}}\mathcal{L}\right)^\top \mathbf{A}^{(1)}$
- ∇ **POST-ACT:** $\frac{\partial \mathcal{L}}{\partial a_j^{(1)}} = \sum_{k=1}^{D^{(2)}} \frac{\partial \mathcal{L}}{\partial z_k^{(2)}} W_{jk}^{(2)}$
  - $\nabla_{\mathbf{a}^{(1)}}\mathcal{L} = \left(W^{(2)}\right)^\top\left(\nabla_{\mathbf{z}^{(2)}}\mathcal{L}\right)$
  - $\nabla_{\mathbf{A}^{(1)}}\mathcal{L} = \left(\nabla_{\mathbf{Z}^{(2)}}\mathcal{L}\right)\mathbf{W}^{(2)}$
- ∇ **PRE-ACT:** $\frac{\partial \mathcal{L}}{\partial z_j^{(1)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(1)}}\text{ReLU}'\left(z_j^{(1)}\right)$
  - $\nabla_{\mathbf{z}^{(1)}}\mathcal{L} = \left(\nabla_{\mathbf{a}^{(1)}}\mathcal{L}\right) \odot \text{ReLU}\left(\mathbf{z}^{(1)}\right)$
  - $\nabla_{\mathbf{Z}^{(1)}}\mathcal{L} = \left(\nabla_{\mathbf{A}^{(1)}}\mathcal{L}\right) \odot \text{ReLU}'\left(\mathbf{Z}^{(1)}\right)$
- ∇ **BIAS:** $\frac{\partial \mathcal{L}}{\partial b_j^{(1)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(1)}}$
  - $\nabla_{\mathbf{b}^{(1)}}\mathcal{L} = \nabla_{\mathbf{z}^{(1)}}\mathcal{L}$, $\nabla_{\mathbf{b}^{(1)}}\mathcal{L} = \left(\nabla_{\mathbf{Z}^{(1)}}\mathcal{L}\right)^\top \mathbf{1}$

6.2 Vectorization Utilities

Pay attention to the dimensionality.

- $\sum_{i=1}^{P} \mathbf{m}_i u_i = M\mathbf{u}$
- $\alpha_i = \mathbf{m}_i^\top \mathbf{u} \Rightarrow \alpha = M\mathbf{u}$
- $\sum_{i=1}^{P} u_i v_i \mathbf{m}_i = M^\top(\mathbf{u} \odot \mathbf{v})$
- $\sum_{i=1}^{P}\sum_{j=1}^{Q} u_i v_j m_{ij} = \mathbf{u}^\top M\mathbf{v}$
- $\sum_{i=1}^{Q}\sum_{k=1}^{Q} u_i m_{ik} u_k = \mathbf{u}^\top M\mathbf{u}$
- $\sum_{i=1}^{P}\sum_{j=1}^{Q} x_i y_j z_{ij} = \mathbf{x}^\top Z\mathbf{y}$
- $\sum_{i=1}^{P} y_i \mathbf{z}_i x_i = Z^\top(\mathbf{x} \odot \mathbf{y})$

Sum $\rightarrow\leftarrow$ is $X\mathbf{1}_D$. Sum $\downarrow$ is $\mathbf{1}_N^\top X$. $\mathbf{u} \odot \mathbf{v} = \text{diag}(\mathbf{u})\,\mathbf{v}$.
Row-wise mult: $\text{diag}(\mathbf{u})\,X$, col: $X \cdot \text{diag}(\mathbf{v})$;
$\begin{bmatrix} -\mathbf{z}^\top W- \\ - \\ - \end{bmatrix} = ZW$

7 BV Decomposition

- Bias: model's inability to capture true relationship.
- Variance: Model's sensitivity to random fluctuations. Resample from training set and model changes by a lot
- Bayes error: Inherent noise in the problem

First: $\mathbb{E}_{\mathbf{x},t,\mathcal{D}}\left[(f(\mathbf{x})-t)^2\right]$

- $= \mathbb{E}\left[\left(f(\mathbf{x}) - \bar{f}(\mathbf{x}) + \bar{f}(\mathbf{x}) - t\right)^2\right]$
- $= \mathbb{E}\left[\left(f(\mathbf{x}) - \bar{f}(x)\right)^2\right] + 0 +$
- $\mathbb{E}\left[\left(\bar{f}(\mathbf{x}) - t\right)^2\right]$

Then: $\mathbb{E}_{\mathbf{x},t,\mathcal{D}}\left[\left(\bar{f}(\mathbf{x}) - t\right)^2\right]$

- $= \mathbb{E}\left[\left(\bar{f}(\mathbf{x}) - \bar{y}(\mathbf{x}) + \bar{y}(\mathbf{x}) - t\right)^2\right]$
- $= \mathbb{E}\left[\left(\bar{f}(\mathbf{x}) - \bar{y}(\mathbf{x})\right)^2\right] +$
- $\mathbb{E}\left[(\bar{y}(\mathbf{x}) - t)^2\right]$

7.1 Bagging

For random forests: using randomization aims to reduce correlation among trees so that their prediction errors are less likely to align. As a result, the ensemble achieves a greater reduction in variance and better generalization.

- $V[y] = V\left[\frac{1}{m}\sum_{i=1}^m y_i\right]$
- $= \frac{1}{m^2} V\left[\sum_{i=1}^m y_i\right]$
- $= \frac{1}{m^2}\sum_{i=1}^m V\left[y_i\right] = \frac{1}{m} V\left[y_i\right]$

8 MLE

The denominator is the same for all classes and scales each numerator by the same amount.

- $p\left(x_j^{(i)} \mid c^{(i)}\right) = \left(p\left(x_j^{(i)} \middle| c^{(i)} = 1\right)\right)^{c^{(i)}} \times$

$$\left(p\left(x_j^{(i)} \middle| c^{(i)} = 0\right)\right)^{1-c^{(i)}}$$

- $\theta_{j1} = \frac{\sum_{i=1}^N c^{(i)} x_j^{(i)}}{\sum_{i=1}^N c^{(i)}}$
- $\theta_{j0} = \frac{\sum_{i=1}^N (1-c^{(i)}) x_j^{(i)}}{\sum_{i=1}^N (1-c^{(i)})}$
- MAP: Multiply by **every** possible parameter. Preferable as it prevents overfitting to limited data, allows us to incorporate any prior belief about the parameter.

9 GDA (D is feat. count)

$$\mathcal{N}(\mathbf{x};\mu,\Sigma) = \frac{1}{(2\pi)^{\frac{D}{2}}|\Sigma|^{\frac{1}{2}}}\exp\left(-\frac{1}{2}\tilde{\mathbf{x}}^\top\Sigma^{-1}\tilde{\mathbf{x}}\right)$$

10 K-Means

Update assignments, update centers. D features, N data points, K clusters, $\vec{m}_k \in \mathbb{R}^D$ denotes the center of cluster k, $r_k^{(i)} \in \{0,1\}$ denotes i is assigned to cluster k w/ center $\vec{m}_k$. Hence, $\vec{r}^{(i)}$ is one-hot. Global objective: $\mathcal{J}\left(\{\vec{m}_k\}, \{\vec{r}^{(i)}\}\right) =$
$\min_{\{\vec{m}_k\},\{\vec{r}^{(i)}\}} \sum_{i=1}^N \sum_{k=1}^K r_k^{(i)} \left\|\vec{m}_k - \vec{x}^{(i)}\right\|^2$

Assignment step: for each point $\vec{x}^{(i)}$, find $\vec{r}^{(i)}$ (just brute-force all possible assignments):
$\min_{\vec{r}^{(i)}} \sum_{k=1}^K r_k^{(i)} \left\|\vec{m}_k - \vec{x}^{(i)}\right\|^2$

Center step (change centers): for center $\vec{m}_k$ is
$\arg\min_{\vec{m}_k} \sum_{i=1}^N r_k^{(i)} \left\|\vec{m}_k - \vec{x}^{(i)}\right\|^2 \Rightarrow \vec{m}_k = \frac{\sum_{i=1}^N r_k^{(i)}\vec{x}^{(i)}}{\sum_{i=1}^N r_k^{(i)}}$

We stop K-means when the assignments no longer change → we have converged to a local minimum. **Local minimum convergence is guaranteed:** each step never increases the objective, there's a finite no. of cluster assignments, algorithm must stop after finite no. of iterations. **If local minima is bad, restart with random cluster centers.**

11 EM

Soft K-means, clusters don't need to be a circle (allows elliptical clusters, and clusters are of a flexible size). A GMM represents $p(\vec{x})$ a mixture (weighted combination of multiple gaussian distributions) of K gaussian components:

$$p(\vec{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}|\mu_k, \Sigma_k)$$

Where $\sum_{k=1}^K \pi_k = 1$, $\pi_k \geq 0$, $\forall k$

Generating data: pick $z^{(i)} \sim \text{Categorical}(\vec{\pi})$, determine features $\vec{x}^{(i)}$: $\vec{x}^{(i)} \mid z^{(i)} \sim \mathcal{N}\left(\vec{x}\middle|\vec{\mu}_{z^{(i)}}, \vec{\Sigma}_{z^{(i)}}\right)$. Hence, generative model is
$p(\vec{x}) = \sum_{k=1}^K \pi_k \mathcal{N}\left(\vec{x}\middle|\vec{\mu}_k, \vec{\Sigma}_k\right)$. $z^{(i)}$ is a latent variable and indicates which Gaussian component generates $\vec{x}^{(i)}$.

EM Algorithm, D features, N data points, K clusters, params: $\vec{\theta} = \left\{\pi_k, \vec{\mu}_k, \vec{\Sigma}_k, k = 1\ldots K\right\}$. Each cluster k Has a mixing weight, mean, and covariance.
$r_k^{(i)} \in [0,1]$ is the responsibility that components k takes for explaining point i (latent variable $z^{(i)}$ indicates that generated point i
Max this likelihood:
$\log p\left(\mathcal{D}\middle|\vec{\theta}\right) = \sum_{i=1}^N \log\left(\sum_{k=1}^K \pi_k \mathcal{N}\left(\vec{x}^{(i)}\middle|\vec{\mu}_k, \vec{\Sigma}_k\right)\right)$
GMMs are universal density approximators. With MLE, training likelihood can grow unbounded, hence **there could be a tiny-variance gaussian on one training example.** This is overfitting. Hence, this is a two-step descent

Expectation step: Compute the responsibility each cluster k takes for explaining point i

- $r_k^{(i)} = p\left(z^{(i)} = k\middle|\vec{x}^{(i)}, \vec{\theta}\right) = \frac{p(z^{(i)}=k)p(\vec{x}^{(i)}|z^{(i)}=k)}{p(\vec{x}^{(i)})}$
- $= \frac{\pi_k \mathcal{N}(\vec{x}|\vec{\mu}_k, \vec{\Sigma}_k)}{\sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}|\vec{\mu}_k, \vec{\Sigma}_k)}$

Maximization step: Maximizes the probability that each cluster generates the data points assigned to it

- $\pi_k = \frac{1}{N}\sum_{i=1}^N r_k^{(i)}$, $\vec{\mu}_k = \frac{\sum_{i=1}^N r_k^{(i)}\vec{x}^{(i)}}{\sum_{i=1}^N r_k^{(i)}}$
- $\vec{\Sigma}_k = \frac{\sum_{i=1}^N r_k^{(i)}\left(\vec{x}^{(i)} - \vec{\mu}_k\right)\left(\vec{x}^{(i)} - \vec{\mu}_k\right)^T}{\sum_{i=1}^N r_k^{(i)}}$

12 PCA

Where $\hat{\mathbf{x}} = \mathbf{U}\mathbf{z} + \mu = \mathbf{U}\mathbf{U}^\top(\mathbf{x} - \mu)$

- Projecting & reconstructing a 2D vector $\mathbf{x} \in \mathbb{R}^2$ onto $\mathbf{u} \in \mathbb{R}^2$: $\hat{\mathbf{x}} = \underbrace{\left((\mathbf{x} - \mu)^\top \mathbf{u}\right)}_{\text{mag}} \cdot \underbrace{\mathbf{u}}_{\text{dir}}$
  - The **projection** step is just $z = \tilde{\mathbf{x}}^\top \mathbf{u} = (\mathbf{x} - \mu)^\top \mathbf{u}$
  - The **reconstruction** step is $\hat{\mathbf{x}} = \mathbf{u}z + \mu$
- The **representation** of $\mathbf{x}$ in basis $\mathbf{U}$ is $\mathbf{z} = \mathbf{U}^\top \mathbf{x}$
- Reconstruction error $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$
- Before PCA, data is best centered since variance is about how data varies around mean, not origin. $\tilde{\mathbf{x}}^{(i)} = \mathbf{x}^{(i)} - \mu$
- Optimal basis (both are equivalent):
  - Max recons. Var.: $\max \frac{1}{N}\sum_{i=1}^N \left\|\hat{\mathbf{x}}^{(i)}\right\|^2$
  - Min recons. Err.: $\min \frac{1}{N}\sum_{i=1}^N \left\|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\right\|^2$

Why are the two conditions above equivalent?

- Prove that $\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}$ and $\hat{\mathbf{x}}^{(i)}$ are orthogonal.
  - Since $\hat{\mathbf{x}}^{(i)}$ is an orthogonal projection of $\mathbf{x}^{(i)}$ onto subspace S, $\left(\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\right)$ is orthogonal to every vector in S. The vector $\hat{\mathbf{x}}^{(i)}$ lies entirely in S. Therefore, $\left(\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\right)$ is orthogonal to $\hat{\mathbf{x}}^{(i)}$.
- By the Pythagorean theorem, $\left\|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\right\|^2 + \left\|\hat{\mathbf{x}}^{(i)}\right\|^2 = \left\|\mathbf{x}^{(i)}\right\|^2$
- Averaging over data points, we obtain: $\frac{1}{N}\sum_{i=1}^N \left\|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\right\|^2 + \frac{1}{N}\sum_{i=1}^N \left\|\hat{\mathbf{x}}^{(i)}\right\|^2 = \frac{1}{N}\sum_{i=1}^N \left\|\mathbf{x}^{(i)}\right\|^2$

12.1 Computing PCA Subspace

Given $\mathbf{x}^{(i)} \in \mathbb{R}^D$, compute $\mathbf{z}^{(i)} \in \mathbb{R}^K$, $K < D$ that preserves as much variance as possible.

1. Center data: $\tilde{\mathbf{x}}^{(i)} = \mathbf{x}^{(i)} - \mu$
2. Compute cov: $\Sigma = \frac{1}{N}\sum_{i=1}^N \left(\tilde{\mathbf{x}}^{(i)}\right)\left(\tilde{\mathbf{x}}^{(i)}\right)^\top$
   a. Cov mat. Captures how much data varies in each direction & how variations in diff. dims relate to each other
3. Find principal components: decompose $\Sigma = \mathbf{Q}\tilde{\ }\mathbf{Q}^\top$, $\mathbf{Q} = [\mathbf{u}_1, \mathbf{u}_2 \ldots]$ (eigenvectors), $\Lambda = \text{diag}(\lambda_1, \ldots)$ (eigenvalues). Σ is PSD ⇒ eigenvals real, NN, $\mathbf{Q}^{-1} = \mathbf{Q}^T$. Then: sort eigenvalues in descending order, select top k corresponding eigenvectors to form $\mathbf{U}_k = [\mathbf{u}_1 \ldots \mathbf{u}_k]$. U_k span the subspace that maxes proj. var or min. recons err

12.2 Shall Data Be Normalized

(For variance only, always normalize the mean).
DO when: Features have diff. units, features have vastly diff. scales, want equal influence from all features
DON'T when: all features same unit, variance differences are meaningful

12.3 The Good, The Bad

The good:

- Visualize data in 2D/3D space.
- Helps with the curse of dimensionality.
- Eliminate noise or irrelevant info from data.
- Extract new, more informative features.

Limits:

- Information loss
- Sensitive to feature scales.
- Challenging to interpret the meanings of the principal components
- Linear transformation only

13 Ethics

COMPAS is the recidivism prediction tool. Fairness:

- **Individual fairness:** fair if it treats people who are **relevantly similar**, similarly
- **Group fairness:** different demographic groups should receive similar overall outcomes
  - e.g. fair if men/women have similar TPR or FPR
- **Relevant similar:** Probably intent, need, consent, risk. Unlikely: race, ethnicity, arbitrary traits.

Sycophancy and Manipulation

- **Noggle:** Manipulative action is the intentional attempt to get someone's **beliefs, desires, or emotions** to violate **their** norms or ideals, from the perspective of the manipulator. (*Influence* aimed at making people act against what they themselves think is right)
- **Sussur, Roessler, Nissenbaum (SNR)**: Causing someone to make non-ideal decisions is sometimes manipulative, but it is not manipulative in every case. Manipulation is imposing a hidden or covert influence on another person's decision-making. *i.e.* influence ≠ manipulation unless emotional pressure, deceptive framing, or psych. Tactics are used
- SNR is about the mechanism (is it hidden, does it bypass awareness)? Noggle is about the goal: is the influencer trying to steer someone away from their own values?